

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Wed 8 July 2015 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

NAME:

MATRICOLA:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the following regular expressions R_1 and R_2 , over the two-letter alphabet $\Sigma = \{ a, b \}$:

$$R_1 = (a a \mid b)^* \qquad R_2 = \Sigma^* a b \Sigma^*$$

Answer the following questions:

- (a) Draw the two deterministic automata A_1 and A_2 equivalent to the two regular expressions R_1 and R_2 , by using the Berry-Sethi method at least for A_2 .
- (b) Check if the two automata A_1 and A_2 are minimal, and if necessary minimize them.
- (c) Find a deterministic automaton A_3 equivalent to the regular expression R_3 below, extended with the intersection operator and the complement one ($\neg$):

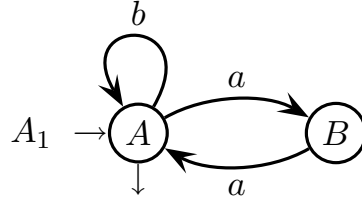
$$R_3 = R_1 \cap \neg R_2$$

and if necessary clean the automaton A_3 by removing the useless states.

- (d) (optional) Find, in a systematic way and in any case justifying your answer, a regular expression R_4 that is equivalent to the extended regular expression R_3 and that contains only the standard operators of concatenation, union and star (or cross).
-

Solution

(a) Automaton A_1 is really straightforward:

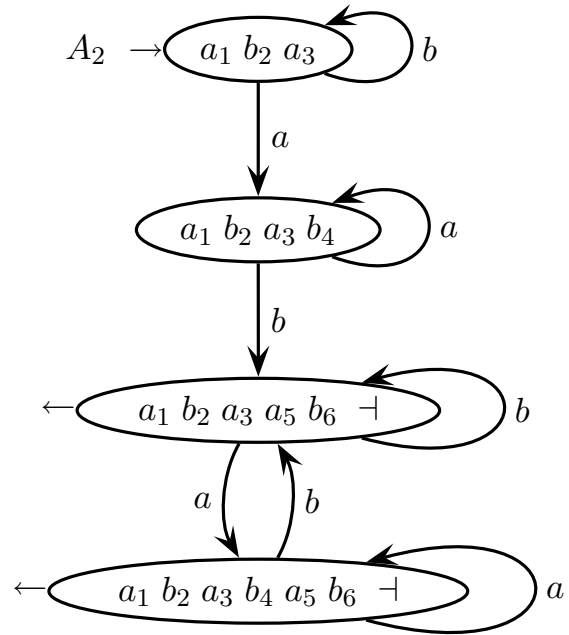


and is deterministic. Berry-Sethi for automaton A_2 :

$$R_{2\#} = (a_1 \mid b_2)^* a_3 b_4 (a_5 \mid b_6)^*$$

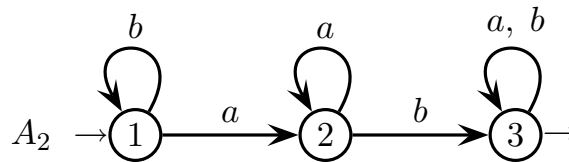
Initials and followers:

<i>initials</i>	$a_1 \ b_2 \ a_3$
<i>generators</i>	<i>followers</i>
a_1	$a_1 \ b_2 \ a_3$
b_2	$a_1 \ b_2 \ a_3$
a_3	b_4
b_4	$a_5 \ b_6 \ \vdash$
a_5	$a_5 \ b_6 \ \vdash$
b_6	$a_5 \ b_6 \ \vdash$



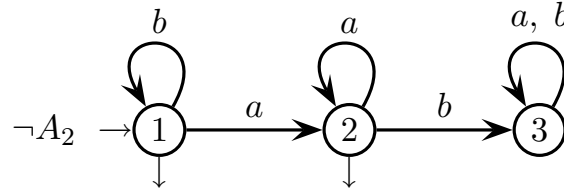
Automaton A_2 is deterministic, of course.

(b) Automaton A_1 is minimal, since the two states are final and non-final, respectively. Automaton A_2 is not minimal: the two final states have the same outgoing labels and the corresponding arcs have the same destination states. So the final states are undistinguishable and can be grouped. The two non-final states are distinguishable by the input letter b , which sends them to a non-final and a to final state. Minimal form of A_2 :

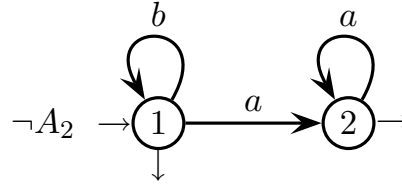


where the final state 3 groups the two original final states.

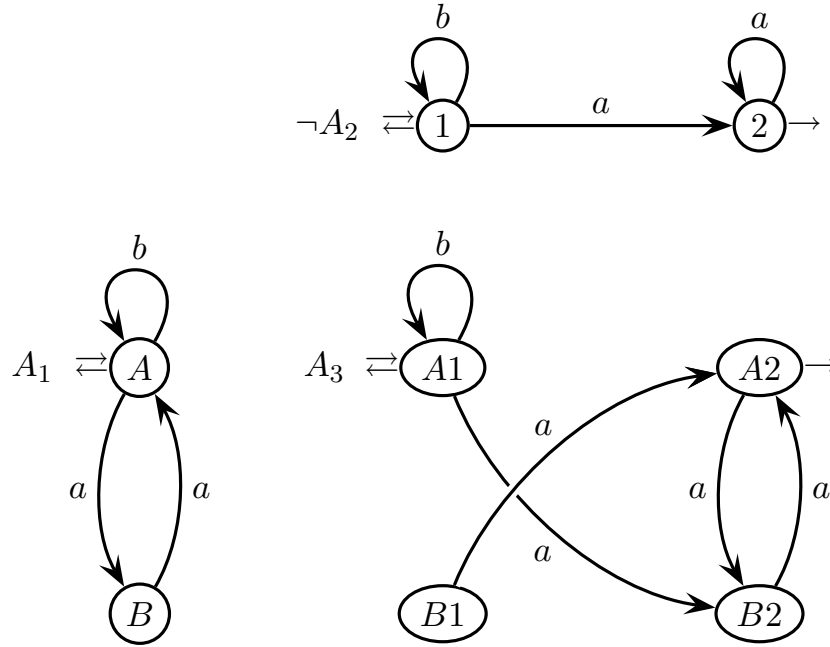
- (c) Notice that automaton A_2 is deterministic and already in the complete form (no need to introduce the error state). So it can be immediately complemented by switching the final and non-final states, as follows:



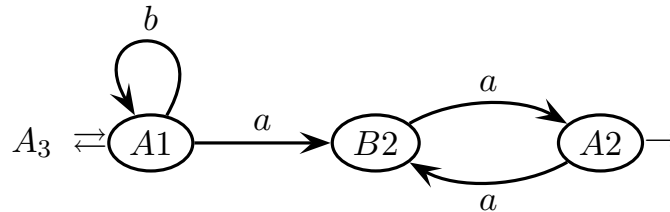
Then it can be cleaned by removing state 3, which has become not post-accessible:



The automaton $\neg A_2$ in clean form is also deterministic and evidently minimal. Now the product automaton A_3 can be constructed:

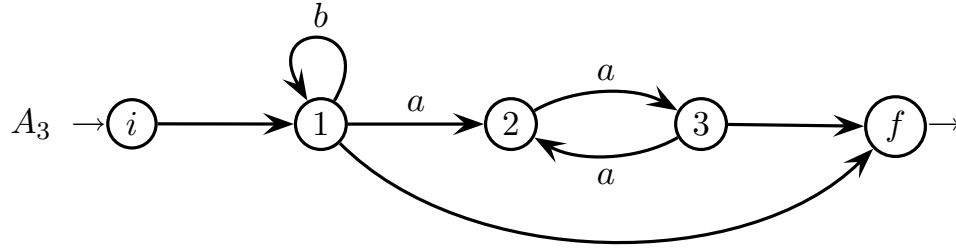


Automaton A_3 is deterministic, of course. State B_1 is not accessible and can be removed. Clean form of A_3 :

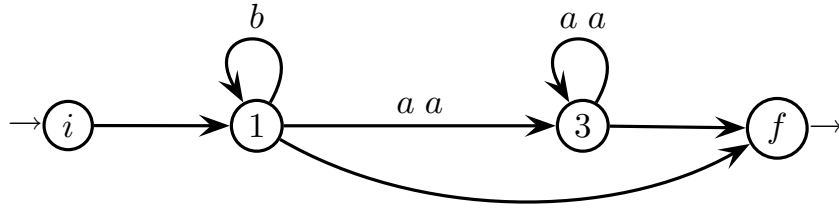


Automaton A_3 in clean form is also evidently minimal.

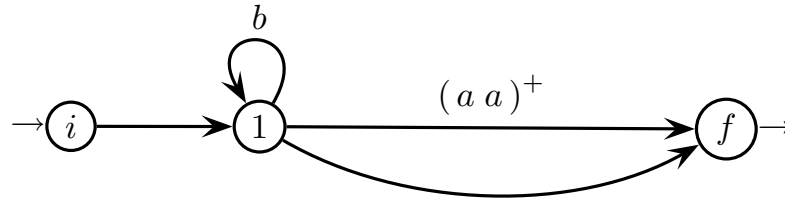
- (d) We can proceed by node elimination (Brzozowski) from automaton A_3 . Unique initial and final states:



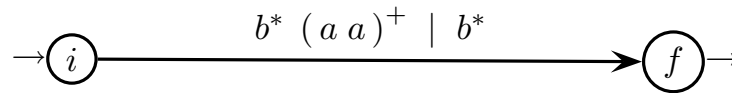
Eliminate node 2:



Eliminate node 3:



Eliminate node 1:



The standard regular expression R_4 equivalent to R_3 is the following:

$$R_4 = b^* (a a)^+ \mid b^* = b^* \left((a a)^+ \mid \varepsilon \right) = b^* (a a)^*$$

Of course, different formulations may exist, as standard as the one above.

2 Free Grammars and Pushdown Automata 20%

1. Answer the following questions:

(a) Write a *BNF* grammar G_1 for the language L_1 below:

$$L_1 = \{ a^i b^j c^k \mid i \neq j \} \quad \text{with } i, j, k \geq 0$$

Write also a similar *BNF* grammar G_2 for the language L_2 below:

$$L_2 = \{ a^i b^j c^k \mid j \neq k \} \quad \text{with } i, j, k \geq 0$$

(b) Based on the grammar G_1 of language L_1 , draw the syntax trees of the two strings:

$$a a b c \quad \text{and} \quad a b b$$

(c) (optional) Considering that the language $L = \{ a^n b^n c^n \mid n \geq 1 \}$ is *not* context-free (i.e., $L \notin CF$), use the fact that $L_1 \in CF$ and $L_2 \in CF$ (as proved by your answer to question (a) above) to argue that the family of context-free languages CF is *not* closed under set complement.

Solution

(a) Here is a *BNF* grammar G_1 (axiom S) of language L_1 , commented aside:

$$G_1 \left\{ \begin{array}{l} S \rightarrow AC \mid BC \quad //L(S) = a^+ a^n b^n c^* \cup a^n b^n b^+ c^* \\ A \rightarrow aA \mid aE \quad //L(A) = a^+ a^n b^n \\ B \rightarrow Bb \mid Eb \quad //L(B) = a^n b^n b^+ \\ C \rightarrow cC \mid \varepsilon \quad //L(C) = c^* \\ E \rightarrow aEb \mid \varepsilon \quad //L(E) = a^n b^n \end{array} \right.$$

with $n \geq 0$. Ratio: A = excess ≥ 1 of a 's, B = excess ≥ 1 of b 's, E = equal number ≥ 0 of a 's and b 's, C = any number ≥ 0 of c 's.

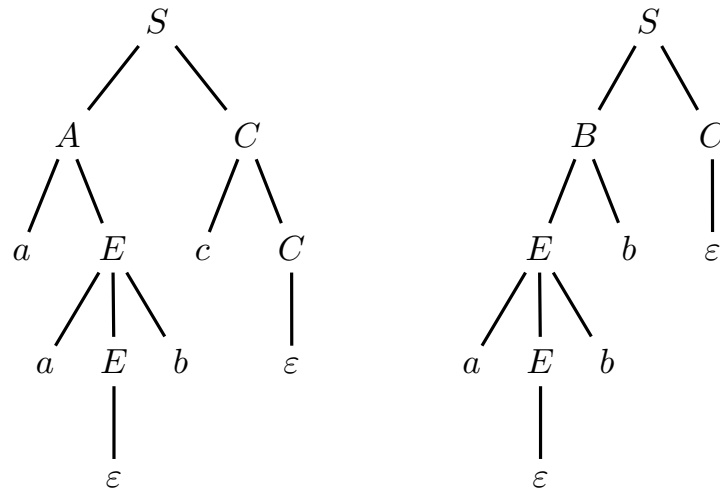
Exploit the same idea as for grammar G_1 . Here is a *BNF* grammar G_2 (axiom S) of language L_2 :

$$G_2 \left\{ \begin{array}{l} S \rightarrow AB \mid AC \\ A \rightarrow aA \mid \varepsilon \\ B \rightarrow bB \mid bE \\ C \rightarrow Cc \mid Ec \\ E \rightarrow bEc \mid \varepsilon \end{array} \right.$$

The ratio is of the same kind as before, swapping the roles of letters a and c .

The explanation of the various nonterminals reasonably proves that both grammars G_1 and G_2 are not ambiguous, though this was not requested.

(b) Here are the syntax trees of grammar G_1 for the strings abc and abb :



which reasonably demonstrate the correctness of grammar G_1 (similarly for G_2).

- (c) Notice that the complement language $\neg L$ is equal to the union of these four component languages, namely:

$$\neg L = \{ \varepsilon \} \cup L_1 \cup L_2 \cup \neg (a^* b^* c^*)$$

The first component provides the empty string, the second and third ones provide the strings of type $a^i b^j c^k$ where at least one of the exponents i , j and k is different from the others, and the fourth one provides the strings where the letters a , b and c are not alphabetically ordered. It is fairly evident that their union makes the complement $\neg L$ of the language L with three equal exponents. These four component languages are all context-free, and actually the first and fourth ones are even regular (the fourth one is the complement of a regular language, so it is regular as well). Hence the complement language $\neg L$ is context-free as well, because the CF family is closed under set union. But if the family CF were closed under set complement, then it would also hold $L \in CF$, which is not the case as recalled before, since L is the language with three equal exponents. Whence the CF family is not closed under set complement.

We give a few more working grammars, for completeness. Here is a different *BNF* grammar for language L_1 :

$$G'_1 \left\{ \begin{array}{l} S \rightarrow E C \\ E \rightarrow a E b \mid A \mid B \\ A \rightarrow a A \mid a \\ B \rightarrow B b \mid b \\ C \rightarrow c C \mid \varepsilon \end{array} \right.$$

which “pumps” the exceeding letters a or b from inside of the structure $a^n b^n$, instead of concatenating them on the left or right thereof, respectively. A similar one can be written for language L_2 .

If it were permitted to use an *EBNF* form, then the grammar of language L_1 could be shortened in this way:

$$G''_1 \left\{ \begin{array}{l} S \rightarrow (a^+ E \mid E b^+) c^* \\ E \rightarrow a E b \mid \varepsilon \end{array} \right.$$

or

$$G'''_1 \left\{ \begin{array}{l} S \rightarrow E c^* \\ E \rightarrow a E b \mid a^+ \mid b^+ \end{array} \right.$$

depending on the mechanism chosen for adding the exceeding letters a or b , and similarly for language L_2 .

2. Consider a data description language that can model the structure of an alphabetical text. The text is modeled by breaking it into substrings, which can be arranged into lists or arrays, and can be possibly given a name. Such a language has these features:

- A text is modeled as a (possibly empty) list of elements.
- A list is enclosed in graph brackets “{” and “}”, and the elements are separated by a comma “,”.
- An element has a name, which is an identifier schematized by the terminal `id`, followed by a colon “:” and then by a value.
- A value may be a string, an array or, recursively, a (possibly empty) list.
- A string is a non-empty sequence of letters (uppercase and lowercase), decimal digits and dots “.”.
- An array is a (non-empty) list of (unnamed) values; the array is enclosed in square brackets “[” and “]”, and the values are separated by a semicolon “;”.

Example: the following text

```
xy.zAabc..0121alphaAX0
```

is modeled by the data description language as:

```
{
  one : xy.zA,
  two : [
    a; bc..0; 12
  ],
  next : {
    inner : [
      1; alpha
    ]
  },
  last : AX0
}
```

Write a grammar, in the extended form (*EBNF*) and not ambiguous, that generates such a data description language.

Solution

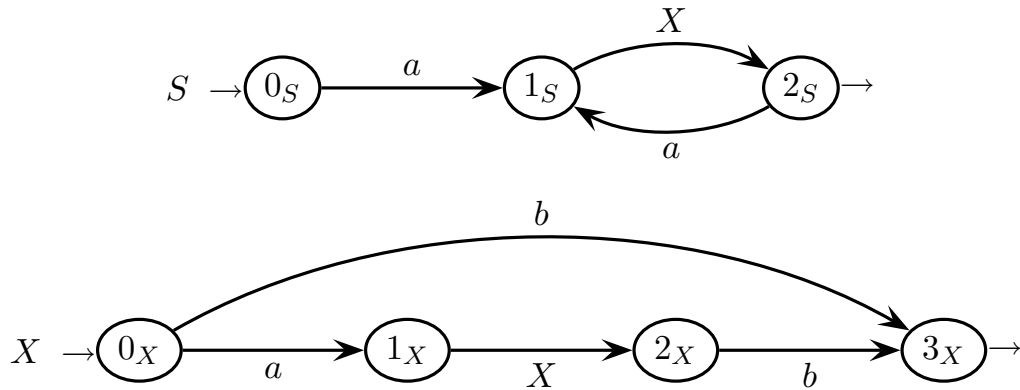
Here is the *EBNF* grammar requested, not ambiguous (axiom TEXT):

$$\begin{aligned}\langle \text{TEXT} \rangle &\rightarrow \text{'\{ '[\langle \text{ELEM_LST} \rangle] '\}' } \\ \langle \text{ELEM_LST} \rangle &\rightarrow \langle \text{ELEMENT} \rangle (\text{' ,' } \langle \text{ELEMENT} \rangle)^* \\ \langle \text{ELEMENT} \rangle &\rightarrow \text{id ' : ' } \langle \text{VALUE} \rangle \\ \langle \text{VALUE} \rangle &\rightarrow \langle \text{STRING} \rangle \mid \langle \text{ARRAY} \rangle \mid \langle \text{TEXT} \rangle \\ \langle \text{STRING} \rangle &\rightarrow ((a \dots z) \mid (A \dots Z) \mid (0 \dots 9) \mid \text{' . ' })^+ \\ \langle \text{ARRAY} \rangle &\rightarrow \text{' [' } \langle \text{VAL_LST} \rangle \text{'] ' } \\ \langle \text{VAL_LST} \rangle &\rightarrow \langle \text{VALUE} \rangle (\text{' ; ' } \langle \text{VALUE} \rangle)^*\end{aligned}$$

Do not confuse between the square brackets [and] when used as metasymbols to mean optionality, and when used as terminal symbols to mean array delimiters; in the latter usage they are enclosed in apices, as customary to clearly specify the terminals. A notation like $(a \dots z)$ is a set containing the elements of an alphabetical (or numerical) interval. Lists may be empty, while arrays may not. The terminal `id` schematizes a whole identifier and, as said before, it is left unexpanded. The grammar is evidently recursive, as lists and arrays may be nested into each other at an arbitrary depth. It is not ambiguous, as reasonably proved by being designed using only unambiguous constructs. Different formulations of this grammar may exist, of course.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, modeled as a net of recursive machines (axiom S):



Answer the following questions:

- (a) Draw the complete *ELR* pilot of grammar G , say if the grammar is of *ELR*(1) type and justify your answer.
- (b) Examine the *ELR* pilot of grammar G , say if the grammar is of *ELL*(1) type and justify your answer. Furthermore, compute all the guide sets (on all the arcs and the exit arrows).
- (c) Simulate the *ELR* (shift-reduce) analysis of the sample string below:

$a \ a \ b \ b \ a \ b$

which belongs to the language $L(G)$. Use the table prepared on the next pages.

- (d) (optional) Simulate the *ELL* (predictive) analysis of the sample string below:

$a \ a \ b \ b \ a \ b$

which belongs to the language $L(G)$. Use the table prepared on the next pages.

simulation table for the *ELR* analysis to be completed
(the number of rows is not significant - $I_0, I_1, \dots$ are the pilot macro-states)

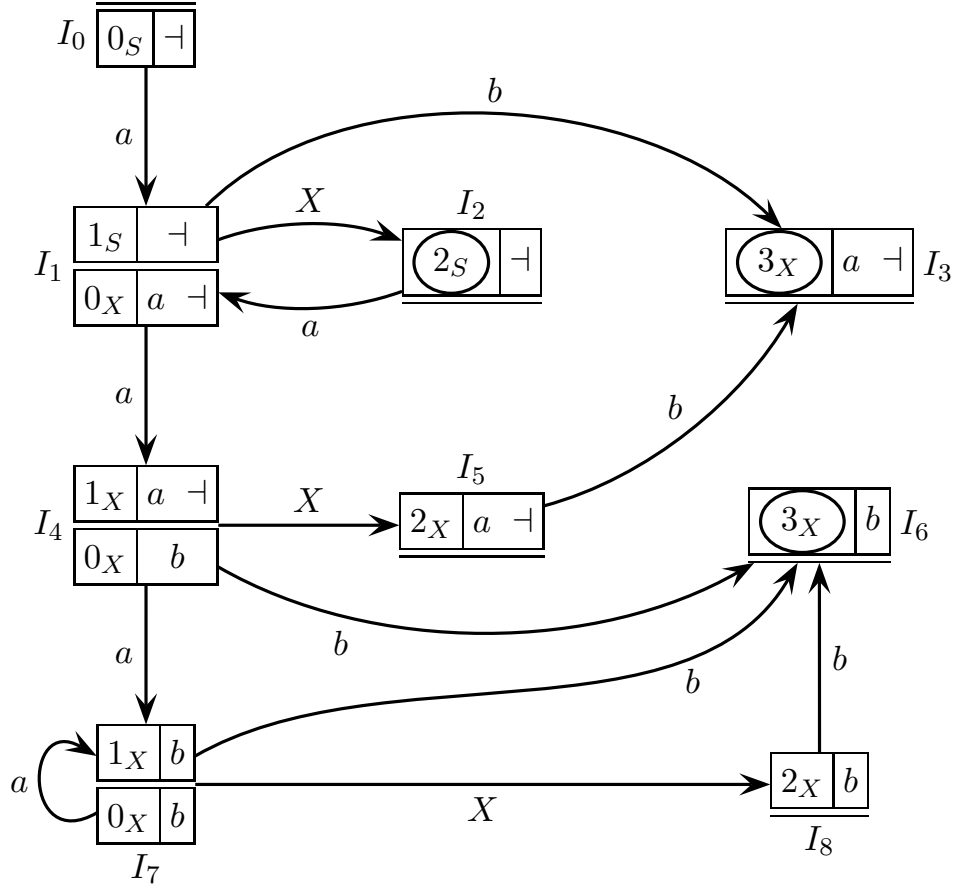
[illegible]

simulation table for the *ELL* analysis to be completed
(the number of rows is not significant)

[illegible]

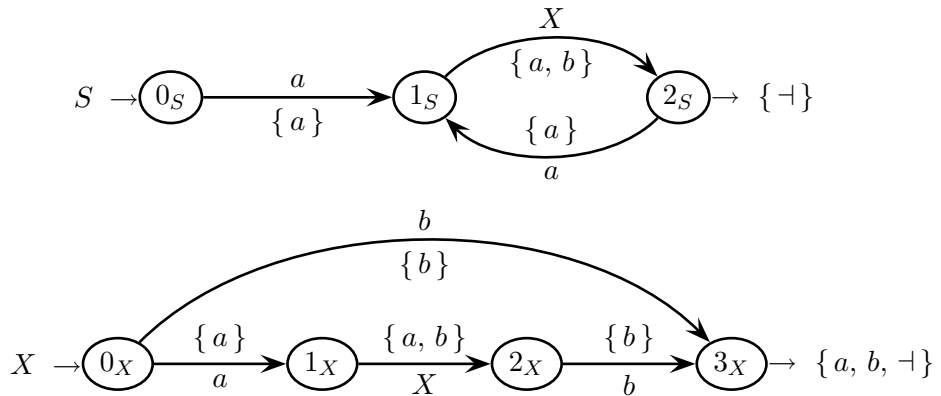
Solution

(a) Here is the complete *ELR* pilot of grammar G , with nine m-states:



There are not any *ELR* conflicts, hence grammar G is of type *ELR*(1).

(b) Condition *ELL*(1) is satisfied: the pilot is of type *ELR*(1), the base of every m-state contains at most one candidate and so the pilot necessarily has the *STP*, and the grammar is not left-recursive. Hence grammar G is of type *ELL*(1). Here are all the guide sets:



Of course, the guide sets at the bifurcation points (i.e., 2_S and 0_X) are disjoint.

- (c) Here is the simulation of the bottom-up pushdown recognizer (*ELR*). The terminator is shown, as it may appear in the reduction look-ahead sets. For simplicity the item identifiers are not shown. Remember that the stack contents consist of interleaved m-states and grammar symbols (nonterminals and terminals):

<i>stack of m-states and symbols</i>	<i>input</i>	<i>move executed</i>
I_0	$a a b b a b \dashv$	initial configuration
$I_0 a I_1$	$a b b a b \dashv$	shifted a
$I_0 a I_1 a I_4$	$b b a b \dashv$	shifted a
$I_0 a I_1 a I_4 b I_6$	$b a b \dashv$	shifted b
$I_0 a I_1 a I_4$	$b a b \dashv$	reduced $b \rightsquigarrow X$
$I_0 a I_1 a I_4 X I_5$	$b a b \dashv$	shifted X
$I_0 a I_1 a I_4 X I_5 b I_3$	$a b \dashv$	shifted b
$I_0 a I_1$	$a b \dashv$	reduced $a X b \rightsquigarrow X$
$I_0 a I_1 X I_2$	$a b \dashv$	shifted X
$I_0 a I_1 X I_2 a I_1$	$b \dashv$	shifted a
$I_0 a I_1 X I_2 a I_1 b I_3$	$\dashv$	shifted b
$I_0 a I_1 X I_2 a I_1$	$\dashv$	reduced $b \rightsquigarrow X$
$I_0 a I_1 X I_2 a I_1 X I_2$	$\dashv$	shifted X
I_0	$\dashv$	reduced $a X a X \rightsquigarrow S$
only the initial m-state	empty tape	reduction to axiom S
<i>ELR</i> acceptance condition true - the string belongs to the language		

Notice that the last reduction $a X a X \rightsquigarrow S$ rolls back twice a loop in the pilot. In principle, the item pointers (not shown here) in the stack m-states should be used to clearly identify the reduction handle. Here the situation is quite simple however, and a little intuition suffices even without the pointer mechanism.

- (d) Here is the simulation of the top-down pushdown recognizer (*ELL*). The terminator is shown, as it may appear in the guide sets. Of course, here the stack alphabet consists simply of the machine states.

<i>stack of machine states</i>	<i>input</i>	<i>move executed</i>
0_S	$a\ a\ b\ b\ a\ b\ \vdash$	initial configuration
1_S	$a\ b\ b\ a\ b\ \vdash$	scanned a
$2_S\ 0_X$	$a\ b\ b\ a\ b\ \vdash$	called X
$2_S\ 1_X$	$b\ b\ a\ b\ \vdash$	scanned a
$2_S\ 2_X\ 0_X$	$b\ b\ a\ b\ \vdash$	called X
$2_S\ 2_X\ 3_X$	$b\ a\ b\ \vdash$	scanned b
$2_S\ 2_X$	$b\ a\ b\ \vdash$	returned from X
$2_S\ 3_X$	$a\ b\ \vdash$	scanned b
2_S	$a\ b\ \vdash$	returned from X
1_S	$b\ \vdash$	scanned a
$2_S\ 0_X$	$b\ \vdash$	called X
$2_S\ 3_X$	$\vdash$	scanned b
2_S	$\vdash$	returned from X
only an axiom final state	empty tape	
<i>ELL</i> acceptance condition true - the string belongs to the language		

4 Language Translation and Semantic Analysis 20%

1. Consider a language L of strings that are two-level lists (non-empty). An atomic element of a list is schematized by the terminal e . Any two consecutive elements (atomic or not) are separated by a comma “,”. The lists at the second level are enclosed in round brackets. Here are a few sample lists:

$e \quad e, e \quad (e) \quad e, (e, e) \quad e, e, (e, e), e \quad (e), (e)$

Answer the following questions:

- (a) Consider a syntactic translation τ_1 of the language L , which flattens the list by replacing every second-level list with a terminal placeholder “ $\perp$ ”. For instance:

$$\tau_1(e, e, (e, e), e) = e, e, \perp, e$$

Write a syntactic translation scheme equivalent to such a translation τ_1 . Use the source grammar G_s below, which generates the language L , and complete it with the destination grammar G_d .

- (b) (optional) Consider a syntactic translation τ_2 of the language L , which swaps the first-level and second-level elements. For instance:

$$\tau_2(e, e, e, (e, e), e) = (e, e, e), e, e, (e)$$

Write a syntactic translation scheme equivalent to such a translation τ_2 . If necessary, you can change the source grammar.

source G_s for (a) (axiom L_1)

destination G_d for (a)

$$L_1 \rightarrow e, L_1$$

$$L_1 \rightarrow e$$

$$L_1 \rightarrow (L_2), L_1$$

$$L_1 \rightarrow (L_2)$$

$$L_2 \rightarrow e, L_2$$

$$L_2 \rightarrow e$$

Solution

(a) Source and destination grammars G_s and G_d (axiom L_1):

<i>source grammar G_s</i>	<i>destination grammar G_d</i>
$L_1 \rightarrow e, L_1$	$L_1 \rightarrow e, L_1$
$L_1 \rightarrow e$	$L_1 \rightarrow e$
$L_1 \rightarrow (L_2), L_1$	$L_1 \rightarrow L_2, L_1$
$L_1 \rightarrow (L_2)$	$L_1 \rightarrow L_2$
$L_2 \rightarrow e, L_2$	$L_2 \rightarrow L_2$
$L_2 \rightarrow e$	$L_2 \rightarrow \perp$

Here the idea is that a placeholder $\perp$ replaces the last element e of a sublist, while all the other elements e (if any) of the sublist, as well as their delimiters (round brackets) and separators (commas) are canceled. Of course, the two grammars G_s and G_d have paired rules with the same nonterminal structure, and the rules of each pair differ only as for the terminals.

The destination grammar G_d has a few alternative formulations, which differ only as for the exact point where to put a placeholder $\perp$. Here are two of them, called G'_d and G''_d . The source grammar is unchanged. Only the last four rules are shown, the first two are the same as in the previous solution:

<i>source gram. G_s</i>	<i>dest. gram. G'_d</i>	<i>dest. gram. G''_d</i>
...	...	...
$L_1 \rightarrow (L_2), L_1$	$L_1 \rightarrow \perp L_2, L_1$	$L_1 \rightarrow L_2 \perp, L_1$
$L_1 \rightarrow (L_2)$	$L_1 \rightarrow \perp L_2$	$L_1 \rightarrow L_2 \perp$
$L_2 \rightarrow e, L_2$	$L_2 \rightarrow L_2$	$L_2 \rightarrow L_2$
$L_2 \rightarrow e$	$L_2 \rightarrow \varepsilon$	$L_2 \rightarrow \varepsilon$

In the previous destination grammar G_d , a placeholder $\perp$ replaces the last element e of a sublist, while in the two new destination grammars G'_d and G''_d it replaces the open or the closed round bracket of a sublist, respectively; all the elements e of a sublist, as well as their remaining delimiters and all their separators, are canceled. Of course, both grammars G'_d and G''_d have the same nonterminal structure as grammar G_s .

- (b) Moving the round brackets from L_2 to L_1 clearly yields a grammar that generates lists with arbitrarily many levels. Thus it is necessary to change the source grammar G_s , to avoid generating arbitrary-level lists. Here is a possible solution (axiom S):

<i>source grammar G_s</i>	<i>destination grammar G_d</i>
$S \rightarrow E$	$S \rightarrow (E)$
$S \rightarrow E, (E)$	$S \rightarrow (E), E$
$S \rightarrow E, (E), S$	$S \rightarrow (E), E, S$
$S \rightarrow (E)$	$S \rightarrow E$
$S \rightarrow (E), S$	$S \rightarrow E, S$
$E \rightarrow e$	$E \rightarrow e$
$E \rightarrow e, E$	$E \rightarrow e, E$

Now a two-level list is generated as a sequence of pieces of type E and (E) , where the nonterminal E is then expanded into a (non-empty) list of atomic elements e . See for instance the sample string below (left):

$$\underbrace{(e, e, e)}_{(E)}, \underbrace{e, e}_E, \underbrace{(e)}_{(E)} \quad \Rightarrow \quad \underbrace{e, e, e}_E, \underbrace{(e, e)}_{(E)}, \underbrace{e}_E$$

Swapping the first-level and second-level lists is thus immediate (see above right). Notice that the source grammar is not ambiguous, thanks to the fact of being designed so that a derivation may not contain two consecutive symbols E (there is always at least one sublist (E) in between). This ensures that the translation is one-valued. Of course, there may be different solutions.

It may be interesting to see an extended (*EBNF*) form of the above solution:

<i>source grammar G'_s</i>	<i>destination grammar G'_d</i>
$S \rightarrow E \left[, (E) \left[, S \right] \right]$	$S \rightarrow (E) \left[, E \left[, S \right] \right]$
$S \rightarrow (E) \left[, S \right]$	$S \rightarrow E \left[, S \right]$
$E \rightarrow e \left[, E \right]$	$E \rightarrow e \left[, E \right]$

which is heavily based on the optionality operator $[]$. Of course, also here the source and destination nonterminal structures must be identical.

2. Lists of nested sequences of dots, such as $\left(\left((\cdot \cdot) \right) \right) (\cdot)$, are used to encode positive integer numbers as sums of powers. For instance, this list:

$$\underbrace{\left(\left(\left(\overbrace{\cdot \cdot}^{2 \text{ dots}} \right) \right) \right)}_{\text{depth 3}} \quad \underbrace{\left(\left(\overbrace{\cdot}^{1 \text{ dot}} \right) \right)}_{\text{depth 2}}$$

encodes the number $3^2 + 2^1 = 9 + 2 = 11$, while the list $((\dots))(((\dots)))$ encodes $2^5 + 4^3 = 32 + 64 = 96$. Hence the number of dots in a sequence represents the exponent, and its nesting depth the base of each power in the sum.

The following grammar generates the list (axiom L):

$$\left\{ \begin{array}{l} L \rightarrow E L \mid E \\ E \rightarrow (E) \mid (P) \\ P \rightarrow . P \mid . \end{array} \right.$$

- (a) Write an attribute grammar that computes the value of the number encoded by the list. The final value is associated to the tree root. It is suggested to use the following attributes:
 - d is the nesting level of each sublist
 - n is the number of dots in each sublist
 - v is the numerical value encoded
- (b) Decorate the syntax tree of the sentence $\left(((\cdot)) \right) ((\cdot))$, by adding to the tree the values of all the attributes. Use the tree prepared on the next page.
- (c) (optional) Determine whether the attribute grammar written is one-sweep and adequately justify your answer.

attributes assigned to be used (complete the specification)

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
	d	integer		nesting level of each sublist
	n	integer		number of dots in each sublist
	v	integer		numerical value encoded

attributes to be added, if any

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>

#	<i>syntax</i>	<i>semantics</i>
---	---------------	------------------

1:	$L_0 \rightarrow E_1 L_2$	
----	---------------------------	--

2:	$L_0 \rightarrow E_1$	
----	-----------------------	--

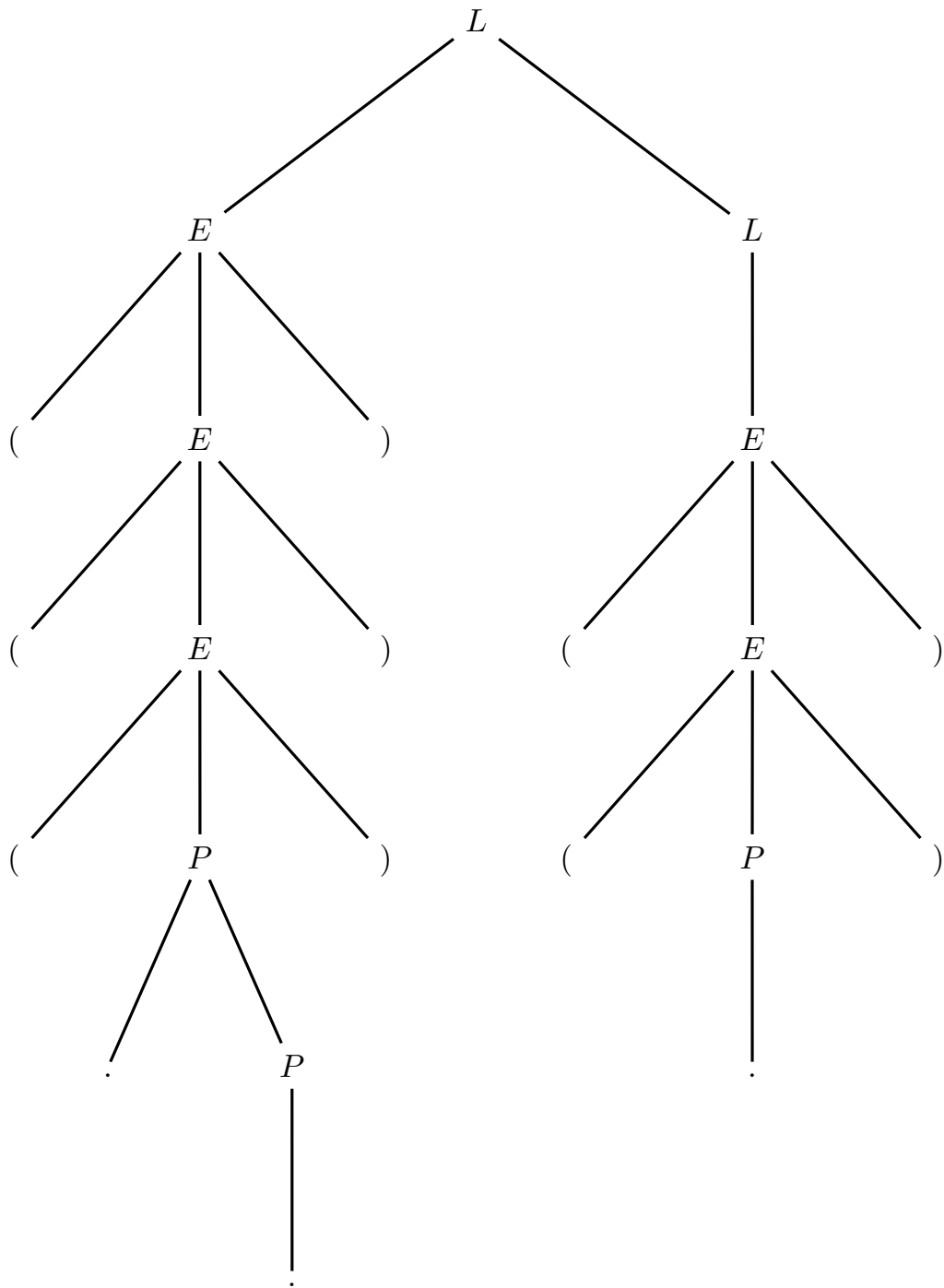
3:	$E_0 \rightarrow (E_1)$	
----	---------------------------	--

4:	$E_0 \rightarrow (P_1)$	
----	---------------------------	--

5:	$P_0 \rightarrow . P_1$	
----	-------------------------	--

6:	$P_0 \rightarrow .$	
----	---------------------	--

syntax tree to be decorated



Solution

(a) Attributes (the assigned ones):

attributes assigned to be used (specification completed)				
<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
right	d	integer	E	nesting level of each sublist
left	n	integer	P	number of dots in each sublist
left	v	integer	L, E	numerical value encoded

No need of more attributes.

Semantic functions (axiom L_0):

#	<i>syntax</i>	<i>semantics</i>	<i>comment</i>
1:	$L_0 \rightarrow E_1 L_2$	$d_1 = 0$ $v_0 = v_1 + v_2$	inherited (initialized) synthesized
2:	$L_0 \rightarrow E_1$	$d_1 = 0$ $v_0 = v_1$	inherited (initialized) synthesized
3:	$E_0 \rightarrow (E_1)$	$d_1 = d_0 + 1$ $v_0 = v_1$	inherited synthesized
4:	$E_0 \rightarrow (P_1)$	$v_0 = (d_0 + 1)^{n_1}$	synthesized
5:	$P_0 \rightarrow . P_1$	$n_0 = n_1 + 1$	synthesized
6:	$P_0 \rightarrow .$	$n_0 = 1$	synthesized (initialized)

This attribute grammar is quite intuitive and deserves few comments: the dots are counted in the rules 5 (initialization) and 6; the levels are counted in the rules 1 and 2 (initialization), 3 and 4; the power is computed in the rule 4 and the summation of the powers is computed in the axiomatic rule 1. The rest of the semantic functions is just for attribute propagation.

Notice: in the rule 4, the inherited attribute d (coming from above in the tree) and the synthesized one n (coming from below in the tree) meet in the formula $(d + 1)^n$ and merge themselves into computing the synthesized attribute v , which climbs the tree and eventually reaches the root with the final result.

If the definition of the attribute d is modified, a different solution exists, purely synthesized. We sketch it here.

Attributes (the assigned ones - with attribute d redefined):

attributes assigned to be used (specification completed)				
<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
left	d	integer	E	<u>reverse</u> nesting level of each sublist
left	n	integer	E, P	number of dots in each sublist
left	v	integer	L	numerical value encoded

Notice that here attribute d specifies the nesting level in reverse order, i.e., the inner-most round brackets are at level 1. This is a change of the exercise text. There is no need of more attributes.

Semantic functions (axiom L_0):

#	<i>syntax</i>	<i>semantics</i>	<i>comment</i>
1:	$L_0 \rightarrow E_1 L_2$	$v_0 = d_1^{n_1} + v_2$	
2:	$L_0 \rightarrow E_1$	$v_0 = d_1^{n_1}$	
3:	$E_0 \rightarrow (E_1)$	$d_0 = d_1 + 1$ $n_0 = n_1$	
4:	$E_0 \rightarrow (P_1)$	$d_0 = 1$ $n_0 = n_1$	initalized
5:	$P_0 \rightarrow . P_1$	$n_0 = n_1 + 1$	
6:	$P_0 \rightarrow .$	$n_0 = 1$	initialized

The difference with respect to the previous solution is that the nesting level is computed upwards instead of downwards. Of course, in this way the nesting level of the internal brackets is different from the standard one. However, at the top the nesting level is correctly known and therefore can be used in the arithmetic computation. This implies that the numbers of dots are propagated upwards at a longer distance, to reach the points where the powers are computed and immediately accumulated. The decorated tree is left to the reader. As this solution is purely synthesized, it is obviously of type one-sweep.